



Hello everyone and wel...
TXT 18.87KB

make notes out of this transcript for me explain all these topics properly
in english with deep concept and code explanation

Machine Learning Foundations: Classification & Model Evaluation — Complete Notes

1. From Regression to Classification

1.1 Recap: Supervised Learning

In supervised learning, we are given training data of the form:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

where each $x_i \in \mathbb{R}^d$ is a **feature vector** (input), and y_i is the **label** (output).

- **Regression:** $y_i \in \mathbb{R}$ (real-valued output)
- **Classification:** $y_i \in \{+1, -1\}$ (discrete/categorical output)

1.2 The Classification Problem — Concrete Example

Suppose you have data about houses:

- **Features:** area, price
- **Goal:** Predict whether the number of rooms is **greater than 3** or **less than or equal to 3**

So the prediction is binary:

- $+1 \rightarrow$ more than 3 rooms
- $-1 \rightarrow$ 3 or fewer rooms

Training data: (x_i, y_i) where $x_i \in \mathbb{R}^d$ and $y_i \in \{+1, -1\}$.

Model: A function $f : \mathbb{R}^d \rightarrow \{+1, -1\}$.

2. Evaluating a Classifier: The Loss Function

2.1 Ideal Case

The best possible model satisfies:

$$f(x_i) = y_i \quad \forall i = 1, \dots, n$$

i.e., every training point is classified correctly.

2.2 Misclassification Loss (0-1 Loss)

In general, the model will make mistakes. The **loss** of a model f is the **fraction of misclassified instances**:

$$L(f) = \frac{1}{n} \sum_{i=1}^n 1[f(x_i) \neq y_i]$$

Where the **indicator function** is:

$$1[f(x_i) \neq y_i] = \begin{cases} 1 & \text{if } f(x_i) \neq y_i \text{ (wrong prediction)} \\ 0 & \text{if } f(x_i) = y_i \text{ (correct prediction)} \end{cases}$$

Interpretation:

- $L(f) = 0 \rightarrow$ perfect classifier
- $L(f) = 0.5 \rightarrow$ random guessing (for balanced binary data)
- $L(f) = 1 \rightarrow$ always wrong

This is also called the **0-1 loss** or **misclassification rate**.

3. Linear Classifiers: The Sign Function Model

3.1 Why Not Linear Regression?

For regression, we used:

$$f(x) = w^T x + b$$

But this outputs real numbers, not $\{+1, -1\}$. We need a binary output.

3.2 The Linear Separator

Apply the **sign function** on top of the linear function:

$$f(x) = \text{sign}(w^T x + b)$$

Where:

$$\text{sign}(z) = \begin{cases} +1 & \text{if } z > 0 \\ -1 & \text{if } z < 0 \\ \text{(convention: } +1 \text{ if } z = 0) \end{cases}$$

This model is called a **linear separator** (or linear classifier / perceptron).

Geometric intuition: The equation $w^T x + b = 0$ defines a **hyperplane** that divides $\mathbb{R}^d$ into two half-spaces:

- One side $\rightarrow +1$
- Other side $\rightarrow -1$

The vector w is **normal** to the hyperplane, and b is the **bias** (offset).

4. Worked Example 1: A Simple 2D Classification Problem

4.1 Dataset

Six 2D data points:

Point	x_1	x_2	Label y
1	0	0	+1
2	1	0	+1
3	0	1	+1
4	4	4	-1
5	3	4	-1
6	4	3	-1

Visualization:

- First three points cluster near the origin $\rightarrow$ **positive class** (red/blue depending on convention)
- Last three points cluster far away $\rightarrow$ **negative class**

4.2 Two Candidate Models

Model f :

$$f(x) = \text{sign}(2 - x_1)$$

Model g :

$$g(x) = \text{sign}(x_1 - 2x_2)$$

4.3 Evaluating Model f

Compute $f(x) = \text{sign}(2 - x_1)$ for each point:

Point	x_1	$2 - x_1$	$f(x)$	True y	Correct?
1	0	2	+1	+1	✓
2	1	1	+1	+1	✓
3	0	2	+1	+1	✓
4	4	-2	-1	-1	✓
5	3	-1	-1	-1	✓
6	4	-2	-1	-1	✓

Loss of f :

$$L(f) = \frac{1}{6} \times 0 = 0$$

Model f is **perfect** on the training data.

Geometric interpretation: f classifies as positive when $2 - x_1 > 0 \Rightarrow x_1 < 2$. So the decision boundary is the vertical line $x_1 = 2$. All points with $x_1 < 2$ are positive; all with $x_1 > 2$ are negative.

4.4 Evaluating Model g

Compute $g(x) = \text{sign}(x_1 - 2x_2)$:

Point	x_1	x_2	$x_1 - 2x_2$	$g(x)$	True y
1	0	0	0	+1*	+1
2	1	0	1	+1	+1
3	0	1	-2	-1	+1
4	4	4	-4	-1	-1
5	3	4	-5	-1	-1
6	4	3	-2	-1	-1

*Convention: $\text{sign}(0) = +1$

Loss of g :

$$L(g) = \frac{1}{6} \times 1 = \frac{1}{6}$$

Conclusion: $L(f) = 0 < L(g) = \frac{1}{6}$, so the learning algorithm would **prefer f over g** .

Geometric interpretation: g classifies as positive when $x_1 - 2x_2 > 0 \Rightarrow x_1 > 2x_2$. The decision boundary is the line $x_1 = 2x_2$. Point $(0, 1)$ lies on the negative side, but it's actually positive — hence the misclassification.

5. Python Code: Implementing the Example

```
python

import numpy as np

# Dataset
X = np.array([
    [0, 0],
    [1, 0],
    [0, 1],
    [4, 4],
    [3, 4],
    [4, 3]
])
y = np.array([+1, +1, +1, -1, -1, -1])

def sign(z):
    """Sign function with convention sign(0) = +1"""
    return np.where(z >= 0, 1, -1)

# Model f: sign(2 - x1)
def f(X):
    return sign(2 - X[:, 0])

# Model g: sign(x1 - 2*x2)
def g(X):
    return sign(X[:, 0] - 2 * X[:, 1])

# Loss function: fraction of misclassified instances
def loss(predictions, true_labels):
    return np.mean(predictions != true_labels)

# Evaluate
f_preds = f(X)
g_preds = g(X)

print("Model f predictions:", f_preds)
print("Model g predictions:", g_preds)
print(f"Loss of f: {loss(f_preds, y)}")
print(f"Loss of g: {loss(g_preds, y)}")

# Output:
# Model f predictions: [ 1  1  1 -1 -1 -1]
# Model g predictions: [ 1  1 -1 -1 -1 -1]
```

Loss of f : 0.0

Loss of g : 0.16666666666666666

6. Worked Example 2: A More Realistic Dataset (House Rooms Prediction)

6.1 The Problem

Predict whether a house has **more than 3 rooms** based on:

- **Area** (feature 1)
- **Price** (feature 2)

Label encoding:

- $-1 \rightarrow 1, 2, \text{ or } 3 \text{ rooms}$
- $+1 \rightarrow \text{more than } 3 \text{ rooms}$

6.2 Dataset (6 houses)

House	Area	Price	True Label
1	8	5	-1
2	9	5	-1
3	12	7	+1
4	13	10	+1
5	15	8	+1
6	14	7	+1

6.3 Three Candidate Models

Model f : $f(x) = \text{sign}(\text{area} - 10)$

Model g : $g(x) = \text{sign}(\text{price} - 6)$

Model h : $h(x) = \text{sign}(\text{price} - 9)$

6.4 Predictions and Losses

House	Area	Price	True y	f (area-10)	g (price-6)
1	8	5	-1	-1 ✓	-1 ✓
2	9	5	-1	-1 ✓	-1 ✓
3	12	7	+1	+1 ✓	+1 ✓
4	13	10	+1	+1 ✓	+1 ✓
5	15	8	+1	+1 ✓	+1 ✓
6	14	7	+1	+1 ✓	+1 ✓

Losses:

- $L(f) = 0/6 = 0$
- $L(g) = 0/6 = 0$
- $L(h) = 3/6 = 0.5$

Conclusion: Both f and g are perfect on training data. The learning algorithm is **indifferent** between them. h is rejected. In practice, a tie-breaking rule or validation data would be needed.

7. Python Code: House Rooms Example

python

```
import numpy as np

# Dataset: [area, price]
X = np.array([
    [8, 5],
    [9, 5],
    [12, 7],
    [13, 10],
    [15, 8],
    [14, 7]
])
y = np.array([-1, -1, +1, +1, +1, +1])

def sign(z):
    return np.where(z >= 0, 1, -1)

# Models
def f(X): # sign(area - 10)
    return sign(X[:, 0] - 10)
```

```
def g(X): # sign(price - 6)
    return sign(X[:, 1] - 6)

def h(X): # sign(price - 9)
    return sign(X[:, 1] - 9)

def loss(preds, true):
    return np.mean(preds != true)

print("Loss f:", loss(f(X), y)) # 0.0
print("Loss g:", loss(g(X), y)) # 0.0
print("Loss h:", loss(h(X), y)) # 0.5
```

8. The Critical Concept: Why We Must NOT Evaluate on Training Data

8.1 The Overfitting Problem — A Counterexample

Consider the earlier 2D dataset (3 positive, 3 negative). Define a **pathological model**:

$$f(x) = \begin{cases} +1 & \text{if } x = (0, 0) \text{ or } (1, 0) \text{ or } (0, 1) \\ -1 & \text{otherwise} \end{cases}$$

On training data: $L(f) = 0$ — perfect!

But is this a good model? No! Consider a new test point like $(0.5, 0.5)$. By intuition (it's near the positive cluster), it should be positive. But this model predicts -1 for it.

8.2 The Core Principle

The goal of machine learning is NOT to do well on data we've already seen. The goal is to generalize — to perform well on unseen future data.

Evaluating on training data gives an **overoptimistic estimate** of model quality. The model can simply **memorize** the training data (like a lookup table) without learning any real pattern.

8.3 The Three Data Splits

Dataset	Purpose	When Used
Training data	Learn the model parameters (e.g., w, b)	During training
Validation data	Select the best model class / hyperparameters	During model selection

Dataset	Purpose	When Used
Test data	Evaluate final model performance	Only once, at the end

8.4 Real-World Analogy

- Train on house prices from 1990–2020
- Evaluate on houses sold in 2021
- Predicting 1990 prices accurately is useless — you already know them!

8.5 Python: Train/Validation/Test Split

python

```
from sklearn.model_selection import train_test_split

# First split: separate test set (20%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Second split: separate validation set from remaining (25% of 80% = 20% total)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42
)

print(f"Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")
```

9. Model Selection vs. Parameter Learning

9.1 Two Distinct Problems

Problem	What is chosen?	Data used	Who decides
Parameter learning	w, b (within a model class)	Training data	Learning algc
Model selection	The model class itself (e.g., linear vs. quadratic)	Validation data	Human + vali

9.2 Example of Model Selection

- **Model class A:** $\text{price} = a \cdot \text{area} + b \cdot \text{rooms} + c \cdot \text{distance}$
- **Model class B:** $\text{price} = a \cdot \frac{\text{area}}{\text{rooms}} + b \cdot \text{distance}^2 + c \cdot \frac{\text{distance}}{\text{area}}$

Both are valid parameterizations. The choice is made by **humans** based on domain knowledge, and the best class is selected using **validation data**.

9.3 The Full Pipeline

```
text

1. Human chooses a collection of model classes (e.g., linear models)
2. Learning algorithm finds best parameters within each class using TRAINING data
3. Human/algorithm selects best model class using VALIDATION data
4. Final model is evaluated on TEST data (never seen before)
```

10. Summary of Key Concepts

Concept	Definition
Classification	Supervised learning where $y \in \{+1, -1\}$
Linear separator	$f(x) = \text{sign}(w^T x + b)$
0-1 Loss	Fraction of misclassified instances
Decision boundary	Hyperplane $w^T x + b = 0$
Overfitting	Model memorizes training data but fails on new data
Training data	Used to learn parameters
Validation data	Used to select model class / hyperparameters
Test data	Used to evaluate final model (never seen during training)
Model selection	Choosing the right model class (human + validation)
Generalization	Performing well on unseen data — the true goal of ML

11. Complete Python Implementation: Linear Classifier from Scratch

```
python

import numpy as np
import matplotlib.pyplot as plt

class LinearClassifier:
```

```
"""
```

```
Linear classifier:  $f(x) = \text{sign}(w^T x + b)$ 
```

```
Trained via perceptron algorithm.
```

```
"""
```

```
def __init__(self, learning_rate=0.01, n_epochs=100):
    self.lr = learning_rate
    self.n_epochs = n_epochs
    self.w = None
    self.b = None

def fit(self, X, y):
    n_samples, n_features = X.shape
    self.w = np.zeros(n_features)
    self.b = 0

    for _ in range(self.n_epochs):
        for i in range(n_samples):
            # Prediction
            y_pred = np.sign(np.dot(self.w, X[i]) + self.b)

            # Perceptron update rule
            if y_pred != y[i]:
                self.w += self.lr * y[i] * X[i]
                self.b += self.lr * y[i]

def predict(self, X):
    return np.sign(np.dot(X, self.w) + self.b)

def loss(self, X, y):
    """0-1 loss: fraction of misclassified instances"""
    preds = self.predict(X)
    return np.mean(preds != y)

# --- Example 1: 2D dataset ---
X = np.array([
    [0, 0], [1, 0], [0, 1],
    [4, 4], [3, 4], [4, 3]
])
y = np.array([1, 1, 1, -1, -1, -1])

clf = LinearClassifier(learning_rate=0.1, n_epochs=100)
clf.fit(X, y)
print("Learned weights:", clf.w)
print("Learned bias:", clf.b)
print("Training loss:", clf.loss(X, y))

# Visualize decision boundary
plt.figure(figsize=(8, 6))
plt.scatter(X[y==1][:,0], X[y==1][:,1], c='blue', label='+1', s=100)
plt.scatter(X[y==-1][:,0], X[y==-1][:,1], c='red', label='-1', s=100)

# Decision boundary:  $w_1x_1 + w_2x_2 + b = 0$ 
x1_vals = np.linspace(-1, 5, 100)
x2_vals = -(clf.w[0]*x1_vals + clf.b) / clf.w[1]
```

```
plt.plot(x1_vals, x2_vals, 'k--', label='Decision boundary')

plt.xlabel('x1')
plt.ylabel('x2')
plt.legend()
plt.title('Linear Classifier Decision Boundary')
plt.grid(True)
plt.show()
```

12. Key Takeaways

1. **Classification** predicts discrete labels ($\{+1, -1\}$), unlike regression which predicts real numbers.
2. **Linear classifiers** use $f(x) = \text{sign}(w^T x + b)$ — a hyperplane separates the classes.
3. **0-1 loss** measures the fraction of misclassified instances.
4. **Never evaluate on training data** — it gives an overoptimistic estimate due to overfitting.
5. **Three data splits:**
 - **Train:** Learn parameters
 - **Validation:** Select model class
 - **Test:** Final evaluation
6. **Model selection** (choosing the model class) is distinct from **parameter learning** (finding w, b within a class).
7. **Generalization** — performing well on unseen data — is the true goal of machine learning.
8. In the next lecture: **Unsupervised learning** (no labels, finding structure in data).

ask me one liner questions to check if i understand all the concepts

Quick Concept Check — One-Liner Questions

Answer these in your head (or out loud). I'll give you the answers after.

Section A: Classification Basics

1. In classification, what values does the label y_i take?
2. What is the key difference between the training data of regression vs. classification?
3. What does a classification model f map from and to?

Section B: Loss Function

4. What does the 0-1 loss measure?
5. What is the loss of a perfect classifier?
6. What does the indicator $1[f(x_i) \neq y_i]$ equal when the prediction is correct?
7. If a classifier gets 3 out of 10 points wrong, what is its loss?

Section C: Linear Classifiers

8. Why can't we directly use $f(x) = w^T x + b$ for classification?
9. What is the formula for a linear separator?
10. What does the equation $w^T x + b = 0$ represent geometrically?
11. What is the convention for $\text{sign}(0)$?
12. In the first example, why did model f have zero loss while g had $1/6$?

Section D: Overfitting & Evaluation

13. Why should you NOT evaluate a model on training data?
14. What is the name of the model that memorizes training points but fails on new data?
15. What is the true goal of machine learning — doing well on training data or on unseen data?
16. What is the difference between validation data and test data?
17. Which data split is used to select the model class (e.g., linear vs. quadratic)?
18. Which data split is used to learn the parameters w and b ?
19. Which data split is used for the final evaluation of the model?
20. In the house price analogy, why is predicting 1990 prices useless?

Section E: Model Selection

21. What is the difference between parameter learning and model selection?
22. Who typically decides the collection of model classes — the learning algorithm or a human?
23. If two models both have zero training loss, how do you choose between them?
24. In the house rooms example, why were both f and g considered equally good?
25. What was wrong with model h in the house rooms example?

Section F: Deeper Understanding

26. Why does evaluating on training data give an "overoptimistic" estimate?
27. What does generalization mean?
28. If a model has 0 loss on training data but performs poorly on test data, what is this called?
29. Why is the 0-1 loss not differentiable, and why does that matter for training?
30. In the pathological model example (point lookup table), why is it a bad model despite zero training loss?

Answers

1. $y_i \in \{+1, -1\}$
2. Labels are discrete ($\{+1, -1\}$) in classification vs. real-valued in regression.
3. From $\mathbb{R}^d$ (feature space) to $\{+1, -1\}$ (labels).
4. The fraction of misclassified instances.
5. Zero.
6. Zero.
7. $3/10 = 0.3$.
8. Because $w^T x + b$ outputs real numbers, not $\{+1, -1\}$.
9. $f(x) = \text{sign}(w^T x + b)$.
10. A hyperplane (decision boundary) separating the two classes.
11. $\text{sign}(0) = +1$.
12. f 's decision boundary ($x_1 = 2$) perfectly separated the classes; g 's boundary ($x_1 = 2x_2$) misclassified point $(0, 1)$.
13. Because the model can memorize training data, giving an overoptimistic estimate of performance.
14. An overfit model.
15. Doing well on unseen data (generalization).
16. Validation is used for model selection; test is used for final evaluation (only once).
17. Validation data.
18. Training data.
19. Test data.
20. Because those prices are already known — the goal is to predict future, unseen prices.
21. Parameter learning finds w, b within a model class; model selection chooses the model class itself.

22. A human (based on domain knowledge/intuition).
23. Use validation data to break the tie.
24. Both had zero training loss — they classified all 6 houses correctly.
25. It had $3/6$ loss — it misclassified 3 houses.
26. Because the model has already seen those points and can memorize them.
27. Performing well on unseen/new data.
28. Overfitting.
29. The sign function has zero gradient almost everywhere, so gradient-based optimization can't be directly applied (need surrogate losses like hinge loss or logistic loss).
30. Because it doesn't learn any pattern — it just memorizes specific points and will fail on any new point, even ones that are intuitively obvious.

How did you do? If you missed any, let me know which ones and I'll explain them in more depth.

you give answers to all the questions

All Questions with Detailed Answers

Section A: Classification Basics

1. In classification, what values does the label y_i take?

Answer: $y_i \in \{+1, -1\}$

In binary classification, labels are discrete and take exactly two values: $+1$ (positive class) or -1 (negative class). This is the fundamental difference from regression where $y_i \in \mathbb{R}$.

2. What is the key difference between the training data of regression vs. classification?

Answer: In regression, labels y_i are real-valued ($y_i \in \mathbb{R}$). In classification, labels are discrete ($y_i \in \{+1, -1\}$).

Both have feature vectors $x_i \in \mathbb{R}^d$, but the output space differs:

- Regression: $f : \mathbb{R}^d \rightarrow \mathbb{R}$
- Classification: $f : \mathbb{R}^d \rightarrow \{+1, -1\}$

3. What does a classification model f map from and to?

Answer: $f : \mathbb{R}^d \rightarrow \{+1, -1\}$

It takes a d -dimensional feature vector as input and outputs either $+1$ or -1 . For example, with house data (area, price), it outputs whether rooms > 3 ($+1$) or ≤ 3 (-1).

Section B: Loss Function

4. What does the 0-1 loss measure?

Answer: The fraction of misclassified instances.

$$L(f) = \frac{1}{n} \sum_{i=1}^n 1[f(x_i) \neq y_i]$$

It counts how many predictions are wrong and divides by the total number of training points.

5. What is the loss of a perfect classifier?

Answer: Zero.

If $f(x_i) = y_i$ for all i , then $1[f(x_i) \neq y_i] = 0$ for all i , so $L(f) = 0$.

6. What does the indicator $1[f(x_i) \neq y_i]$ equal when the prediction is correct?

Answer: Zero.

The indicator equals:

- 0 when $f(x_i) = y_i$ (correct prediction)
- 1 when $f(x_i) \neq y_i$ (wrong prediction)

7. If a classifier gets 3 out of 10 points wrong, what is its loss?

Answer: $3/10 = 0.3$

$$L(f) = \frac{1}{10} \times 3 = 0.3$$

Section C: Linear Classifiers

8. Why can't we directly use $f(x) = w^T x + b$ for classification?

Answer: Because $w^T x + b$ outputs real numbers ($\mathbb{R}$), but classification requires outputs in $\{+1, -1\}$.

We need a function that maps real numbers to discrete labels. The sign function does this.

9. What is the formula for a linear separator?

Answer: $f(x) = \text{sign}(w^T x + b)$

Where:

- $w \in \mathbb{R}^d$ is the weight vector
- $b \in \mathbb{R}$ is the bias
- $\text{sign}(z) = +1$ if $z > 0$, -1 if $z < 0$

10. What does the equation $w^T x + b = 0$ represent geometrically?

Answer: A hyperplane (decision boundary) that separates $\mathbb{R}^d$ into two half-spaces.

- One side: $w^T x + b > 0 \Rightarrow$ predicted $+1$
- Other side: $w^T x + b < 0 \Rightarrow$ predicted -1

In 2D, it's a line. In 3D, it's a plane. In higher dimensions, it's a hyperplane.

11. What is the convention for $\text{sign}(0)$?

Answer: $\text{sign}(0) = +1$

This is a convention (arbitrary choice) to handle the boundary case. We could also define it as -1 , but $+1$ is commonly used.

12. In the first example, why did model f have zero loss while g had $1/6$?

Answer: Model $f(x) = \text{sign}(2 - x_1)$ classified all 6 points correctly.

- Decision boundary: $x_1 = 2$
- All points with $x_1 < 2$ (the three positive points) $\rightarrow +1$ ✓
- All points with $x_1 > 2$ (the three negative points) $\rightarrow -1$ ✓

Model $g(x) = \text{sign}(x_1 - 2x_2)$ misclassified point $(0, 1)$:

- $g(0, 1) = \text{sign}(0 - 2) = \text{sign}(-2) = -1$
- But true label is $+1$ ✗

So $L(g) = 1/6$.

Section D: Overfitting & Evaluation

13. Why should you NOT evaluate a model on training data?

Answer: Because the model has already seen those points and can simply memorize them, giving an overoptimistic estimate of performance.

The model might achieve zero training loss by memorizing specific points without learning any real pattern. On new, unseen data, it would fail badly.

14. What is the name of the model that memorizes training points but fails on new data?

Answer: An overfit model.

It has low training error but high test error. It captures noise rather than the underlying pattern.

15. What is the true goal of machine learning — doing well on training data or on unseen data?

Answer: Doing well on unseen data (generalization).

The goal is to learn patterns that generalize to new, future data points. Training performance is just a means to an end.

16. What is the difference between validation data and test data?

Answer:

Validation Data	Test Data
Used for model selection (choosing model class/hyperparameters)	Used for final evaluation of the chosen model
Can be used multiple times	Used only once, at the very end

Validation Data	Test Data
Helps tune the model	Gives unbiased estimate of final performance

17. Which data split is used to select the model class (e.g., linear vs. quadratic)?

Answer: Validation data.

The validation set helps choose between different model classes (e.g., linear vs. polynomial) or hyperparameters (e.g., learning rate, regularization strength).

18. Which data split is used to learn the parameters w and b ?

Answer: Training data.

The learning algorithm uses training data to find the optimal parameters within a given model class.

19. Which data split is used for the final evaluation of the model?

Answer: Test data.

The test set is never used during training or model selection. It gives an unbiased estimate of how the model will perform on truly unseen data.

20. In the house price analogy, why is predicting 1990 prices useless?

Answer: Because those prices are already known — the goal is to predict future, unseen prices.

If you train on 1990–2020 data, you want to predict 2021+ prices. Evaluating on 1990 data just tells you how well you memorized the past, not how well you'll predict the future.

Section E: Model Selection

21. What is the difference between parameter learning and model selection?

Answer:

Parameter Learning	Model Selection
Finding w, b within a given model class	Choosing the model class itself
Done by the learning algorithm	Done by humans + validation data
Uses training data	Uses validation data
Example: finding best line	Example: choosing line vs. parabola

22. Who typically decides the collection of model classes — the learning algorithm or a human?

Answer: A human (based on domain knowledge/intuition).

For example, a human decides whether to use a linear model, polynomial model, neural network, etc. The learning algorithm then finds the best parameters within that chosen class.

23. If two models both have zero training loss, how do you choose between them?

Answer: Use validation data to break the tie.

Evaluate both models on the validation set and pick the one with lower validation loss. If still tied, use domain knowledge or additional criteria (e.g., simplicity/regularization).

24. In the house rooms example, why were both f and g considered equally good?

Answer: Both had zero training loss — they classified all 6 houses correctly.

- $f(x) = \text{sign}(\text{area} - 10)$: Perfect on training data
- $g(x) = \text{sign}(\text{price} - 6)$: Perfect on training data

On the training data alone, there's no way to distinguish between them. Validation data would be needed.

25. What was wrong with model h in the house rooms example?

Answer: It had $3/6$ loss — it misclassified 3 houses.

$h(x) = \text{sign}(\text{price} - 9)$ predicted -1 for houses 3, 5, and 6, but their true labels were $+1$. Only house 4 had $\text{price} > 9$.

Section F: Deeper Understanding

26. Why does evaluating on training data give an "overoptimistic" estimate?

Answer: Because the model has already seen those points and can memorize them.

A sufficiently complex model can achieve zero training error by memorizing the training set (like a lookup table), even if it hasn't learned any meaningful pattern. On new data, it will fail.

27. What does generalization mean?

Answer: Performing well on unseen/new data.

A model that generalizes well has learned the underlying pattern (not just memorized training points) and can make accurate predictions on data it has never seen before.

28. If a model has 0 loss on training data but performs poorly on test data, what is this called?

Answer: Overfitting.

The model has memorized the training data (including noise) rather than learning the true underlying pattern.

29. Why is the 0-1 loss not differentiable, and why does that matter for training?

Answer: The sign function and indicator function have zero gradient almost everywhere, making gradient-based optimization impossible.

The 0-1 loss is a step function — its derivative is 0 everywhere except at the discontinuity (where it's undefined). This means we can't use gradient descent directly. Instead, we use **surrogate losses** like:

- **Hinge loss:** $\max(0, 1 - y_i(w^T x_i + b))$
- **Logistic loss:** $\log(1 + e^{-y_i(w^T x_i + b)})$

These are differentiable and upper-bound the 0-1 loss.

30. In the pathological model example (point lookup table), why is it a bad model despite zero training loss?

Answer: Because it doesn't learn any pattern — it just memorizes specific points and will fail on any new point, even ones that are intuitively obvious.

The model $f(x) = +1$ only for $(0, 0)$, $(1, 0)$, $(0, 1)$ and -1 otherwise:

- On training data: 0 loss ✓
- On new point $(0.5, 0.5)$: predicts -1 ✗ (should be $+1$ by intuition)

It has learned nothing about the underlying structure. It's just a lookup table.

Summary Table of Key Answers

#	Question	Answer
1	Label values	$\{+1, -1\}$
4	0-1 loss measures	Fraction misclassified
5	Perfect classifier loss	0
9	Linear separator formula	$\text{sign}(w^T x + b)$
10	$w^T x + b = 0$	Hyperplane/decision boundary
13	Why not evaluate on train?	Overoptimistic (memorization)
15	True goal of ML	Generalization to unseen data
17	Model class selection	Validation data
18	Parameter learning	Training data
19	Final evaluation	Test data
21	Parameter vs. model selection	Parameters within class vs. choosing class
28	0 train loss, poor test	Overfitting
29	Why 0-1 loss not differentiable	Zero gradient; need surrogate losses

How did you do? If any answer is still unclear, tell me the question number and I'll explain it with a different example or more detail.